Информационные системы управления данными и документами

ФИО ст.преподавателя: Тесла-Блозва Святослав Александрович

e-mail: teslablozva@mirea.ru

Условия обучения

- По итогам изучения дисциплины проводится экзамен
- В течение семестра необходимо выполнить все задания по плану, который опубликован на Учебном портале online-edu.mirea.ru

План обучения по дисциплине

№	Тема лекции	Содержание
1	Введение в системы электронного документооборота и контейнеризация	Ключевые понятия СЭД, нормативная база. Роль контейнеризации в современных ИС. Docker. Образы, контейнеры, тома, сети. Docker Compose для оркестрации многоконтейнерных приложений. Обзор архитектуры «Атом.Платформы» и ее компонентов.
2	Пользовательский интерфейс с Vaadin	Фреймворк Vaadin. Серверно-ориентированный подход. Компоненты, компоновка. Binder. Связывание данных и валидация. Обработка событий, уведомления. Создание форм для работы с объектами.
3	Работа с данными и Spring Data JPA	Объектно-реляционное отображение (ORM), JPA. Аннотации сущностей, первичные ключи, репозитории. Spring Data JPA. Автоматические CRUD-методы, производные запросы. Интеграция с PostgreSQL. Табличные представления: Vaadin Grid, сортировка, фильтрация.
4	Связанные справочники и настройка интерфейса	Нормализация данных, связи между сущностями. Справочники как отдельные сущности. Выбор из справочника. Стилизация интерфейсов, темы Lumo, CSS-переменные, компактный режим. Требования корпоративного дизайна.
5	Полнотекстовый поиск и OpenSearch	Ограничения SQL-поиска. Инвертированный индекс. OpenSearch: индексы, маппинг, анализаторы (в т.ч. для русского языка). Синхронизация данных PostgreSQL и OpenSearch. Реализация поисковых запросов, ранжирование, подсветка фрагментов.
6	Управление бизнес-процессами и оркестрация на базе Camunda	BPMN 2.0. Основные элементы (события, задачи, шлюзы). Понятие процессной оркестрации. Camunda Platform - архитектура, интеграция с Spring Boot. Пользовательские и сервисные задачи, JavaDelegate. Переменные процесса, запуск и управление процессами.
7	Мониторинг и наблюдаемость корпоративных систем	Метрики, логи, трейсинг. Prometheus. Модель pull, экспортеры, язык PromQL. Grafana. Визуализация, дашборды, алертинг. Сбор метрик.
8	Эксплуатация, масштабирование и сравнение платформ.	Обеспечение отказоустойчивости, резервное копирование, обновление компонентов. Масштабирование микросервисов. Сравнение «Атом.Платформы» с альтернативными СЭД. Перспективы развития. Искусственный интеллект в СЭД, переход на облачные технологии.

Электронный документооборот (ЭДО) и СЭД/ECM-системы

ЭДО — это автоматизированная система, обеспечивающая создание, согласование, подписание и хранение электронных документов, упрощая работу с ними внутри компании и с контрагентами.

Ключевые преимущества ЭДО

- Ускорение бизнес-процессов и повышение прозрачности работы.
- Экономия времени и ресурсов, снижение ошибок и налоговых рисков.
- Удалённая работа и обмен юридически значимыми документами.

Ведущие российские СЭД/ECM

- Directum — полный жизненный цикл документов, поддержка BPM/Workflow, мобильность.
- Globus — гибкая настройка под бизнес-задачи, масштабируемость, развитые отчёты.
- TESSA — высокая скорость, продвинутый веб-клиент, гибкие бизнес-процессы, интеграция.
- Docsvision — модульная структура, мощная интеграция, поддержка MS .NET и SharePoint.
- NauDoc — оптимально для малого и среднего бизнеса, бесплатные лицензии.

Современные тренды рынка

- Импортозамещение и зрелость отечественных решений.
- Внедрение кибербезопасности, ИИ и RPA (роботизация).
- Поддержка мультиканальности (разные устройства и ОС).

Что такое «Атом.Платформа»?

- Технологическое ядро «Атом.Платформы» (набор программных библиотек), предоставляющее средства защиты информации (СЗИ) и работы с данными.
- Является многопользовательской системой для работы со структурированными и неструктурированными данными в сетевой среде.
- Включает сертификат соответствия ФСТЭК России, также находиться в реестре отечественного ПО.
- Также является веб-приложением для администрирования систем, реализованных на базе «Атом.Платформы».
- Применение лучших практик ECM-систем для работы со структурированными и неструктурированными данными (в том числе опыт OpenText Documentum).

Конкурентные преимущества Атом.Контент

Работа с большими данными

Пример внедрения системы ЕОСДО 2.0 на базе Атом.Контент :
110+тыс. пользователей,
объем данных **19+ Tb** БД,
180+ Tb файлового хранилища, **1500 операций в секунду**

Защищенность информации

Ролевая модель
сертификат ФСТЭК (4 уровень доверия, №4641 выдан 29.12.2022 г)
Включено в реестр отечественного ПО



Применение лучших практик для ECM-систем
(в том числе опыт OpenText Documentum)

Команда с экспертизой более 27 лет

в области автоматизации БП и тех.поддержки систем с кол-вом пользователей более 100 000
Численность сотрудников: более 160 сертифицированных специалистов

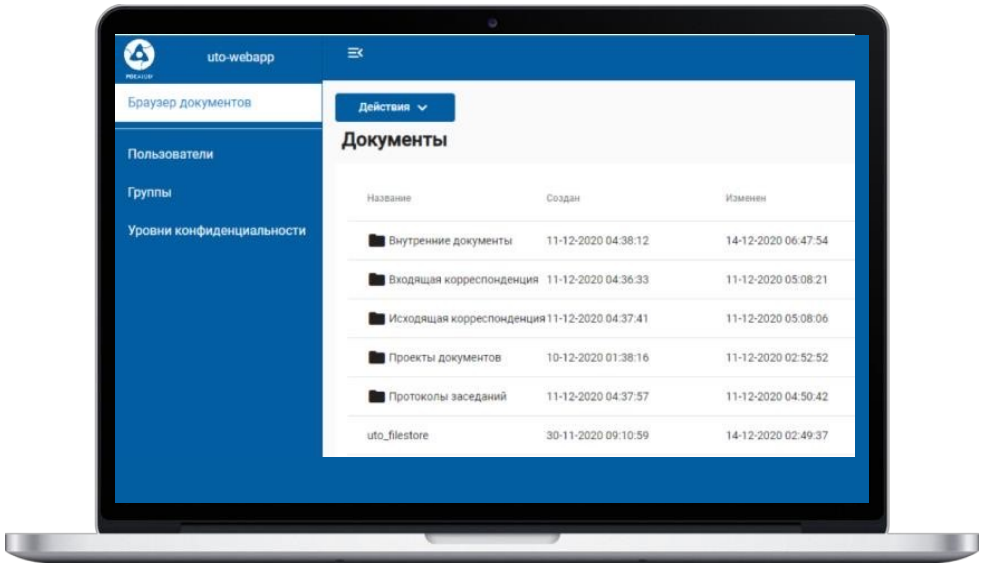
Возможности платформы Атом.Контент

- Управление моделью данных
  (Hibernate, Liquibase)
- Управление контентом (структурированным и не структурированным)
  (файлы, папки, кабинеты)
- Подключение разных типов хранилищ
  (БД, файлы, S3)
- Поддержка заданий
  (Jobs)
- Аутентификация
  (OAuth 2.0, LDAP)
- Аудит
  (Elastic Stack)
- Управление пользователями и группами
- Списки доступов (ACL)
- Разграничение доступа к объектам (ABAC)
- Современный, распространенный технологический стек
  (Java, JavaScript, SQL, CSS, HTML)
- Поддержка REST API (Swagger, OpenAPI 3.0)

Управление администрированием

В состав платформы «Атом.Контент» входит интерфейс администратора, позволяющий управлять объектами системы.

- Управление пользователями и группами
- Управление правами
- Ведение системных справочников
- Управление объектами: репозиторями, папками, документами, ревизитами
- Управление заданиями
- Конфигрирование LDAP, OAuth 2.0



Обзор библиотек Атом.Контент

Атом.Контент – набор библиотек Unifique Technical Objects (UTO)

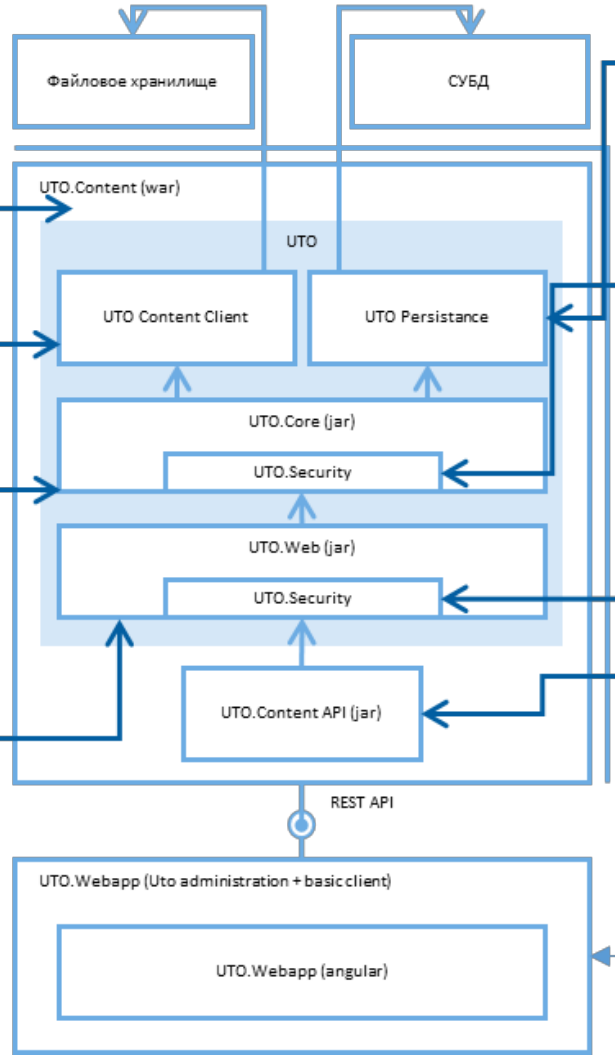
Целевое приложение (UTO.Content), основанное на библиотеках UTO. Используется как подсистема администрирования (настройка авторизации, системные настройки, работа с учетными записями и группами, работа с папками и документами, работа с аудитом)

Адаптер (UTO Content Client), реализующий взаимодействие с файловым хранилищем

Модуль UTO.Core обеспечивает основные функции UTO:

- по логике работы с мета-данными и контентом;
- по обеспечению безопасности;
- по взаимодействию с файловыми хранилищами и СУБД

Модуль UTO.Web обеспечивает логику взаимодействия с UTO.Core посредством web-сессий (на входе имеет http-запросы), выполняет идентификацию и аутентификацию пользователя



Адаптер, реализующий взаимодействие с БД

Логический модуль (UTO.Security), объединяет функции СЗИ, реализуемые в Атом.Контент

Модуль (UTO.Content API) реализует REST API и логику работы с функциями UTO

Приложение прикладного администратора (UTO.WebApp) - пользовательский интерфейс UTO.Content, взаимодействует с REST API. Используется администраторами, также может применяться как пользовательский клиент для работы с документами:

Защита информации

Атом.Контент» имеет встроенные средства защиты информации (СЗИ)

- Идентификация и аутентификация (OAuth 2.0, LDAP)
- Дискреционный доступ к объектам данных (списки доступа ACL)
- Действия над объектами (разрешение/запрет) на основании наборов политик (ABAC)
- Аудит событий
- Администрирование через Веб-интерфейс
- СЗИ «Атом.Контент» сертифицировано ФСТЭК

Функциональная структура

Основные возможности:

- Работа с файлами любых форматов (неструктурированные данные).
- Ввод и управление структурированной информацией.
- Контроль версий
- Разграничение доступа
- Идентификация, аутентификация, авторизация пользователей.

Объектное хранилище данных (ООП):

- Объект мета-данных – хранит структурированную информацию (может быть связан с контентом).
- Контент – файл (всегда имеет связанный объект мета-данных).
- Папка – логическая структура, содержит ссылки на мета-объекты и папки.

Безопасность:

- Модуль безопасности – дискреционная модель (списки доступа).
- Вычислитель ABAC – принятие решений по политикам (разрешить/запретить).
- Интеграция OAuth 2.0 – централизованное управление пользователями через внешний каталог.

Варианты применения платформы Атом.Контент

01

Используется для создания систем,
в которых необходимо хранение
и обработка контента:

- Электронных архивы
- Системы электронного документооборота
- Системы управления знаниями
- Кредитные конвейеры
- Кадровые системы

02

Используется в качестве встраиваемого хранилища контента (документов, файлов) для систем класса:
ERP
CRM
BPM
BI
Порталы

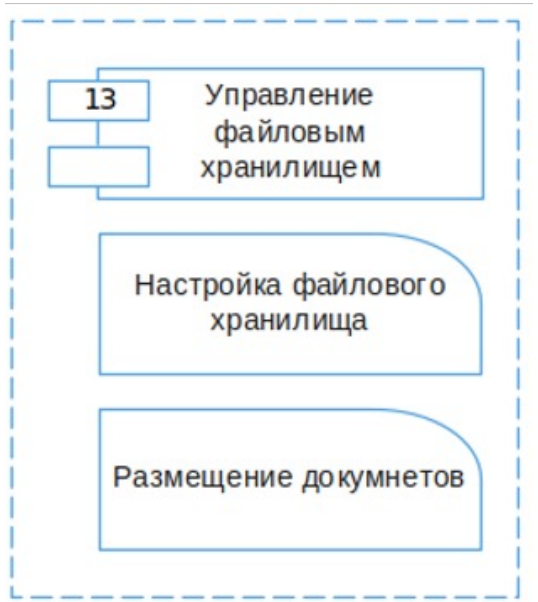
Конструктор форм

- В настоящий момент в процессе реализации находится компонент платформы «Фреймвок для разработки UI», включающий готовую интеграцию с ПО «Атом.Форма»
- Компонент предназначен для быстрой разработки пользовательских интерфейсов приложения с автоматической привязкой к модели данных.
- Компонент реализуется в рамках проекта «Электронный архив».



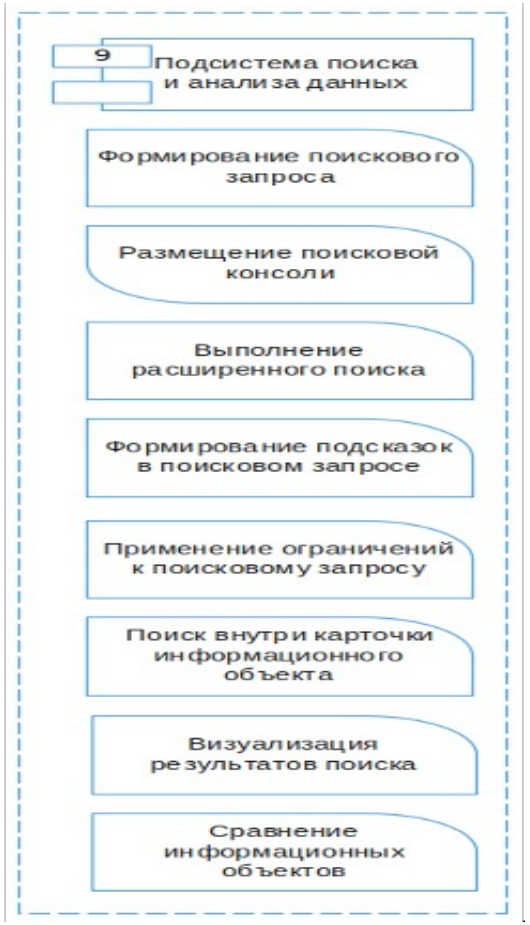
Управление файловым хранилищем

- Компонент входит в состав платформы. Поддерживает различные виды хранилищ (в т.ч. файловые системы, S3, БД).
- Позволяет подключать дополнительные виды хранилищ.
- Поддерживает создание множественных репозиториев.



Подсистема поиска

- Базовый функционал
- Компонент входит в состав платформы.
- Реализован на OpenSearch.
- Обеспечивает поиск по содержанию и реквизитам в системе ЕОСДО-2.0.
- Расширенный функционал (в разработке)
- Готовая интеграция с «Атом.Поиск»



Управление версиями

- Компонент входит в состав ядра платформы.
- Позволяет создавать, удалять и управлять версиями объектов.



Управление ИБ

- Компонент входит в состав ядра платформы.
- Поддерживает аутентификацию Oauth-2.0
- Используется для аутентификации в ЕОСДО-2.0
- Предоставляет возможность распределения
- доступа на уровне каждого объекта.
- Позволяет настраивать доступ на уровне групп и пользователей.
- Позволяет настраивать доступ на основе правел (ABAC).



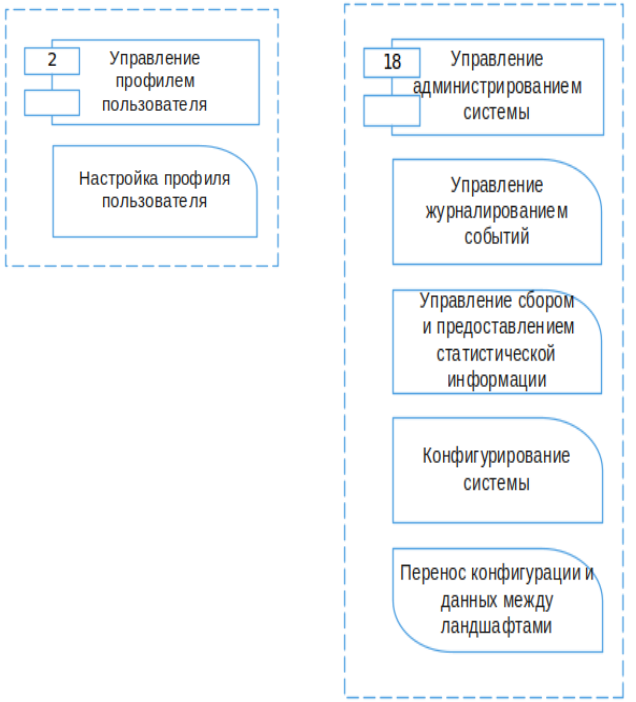
Управление бизнес-процессами

- В настоящий момент в процессе реализации находится компонент платформы «BPM».
- Компонент предназначен для быстрого построения бизнес-процессов при разработки приложений.
- Компонент реализуется в рамках проекта «Архив».

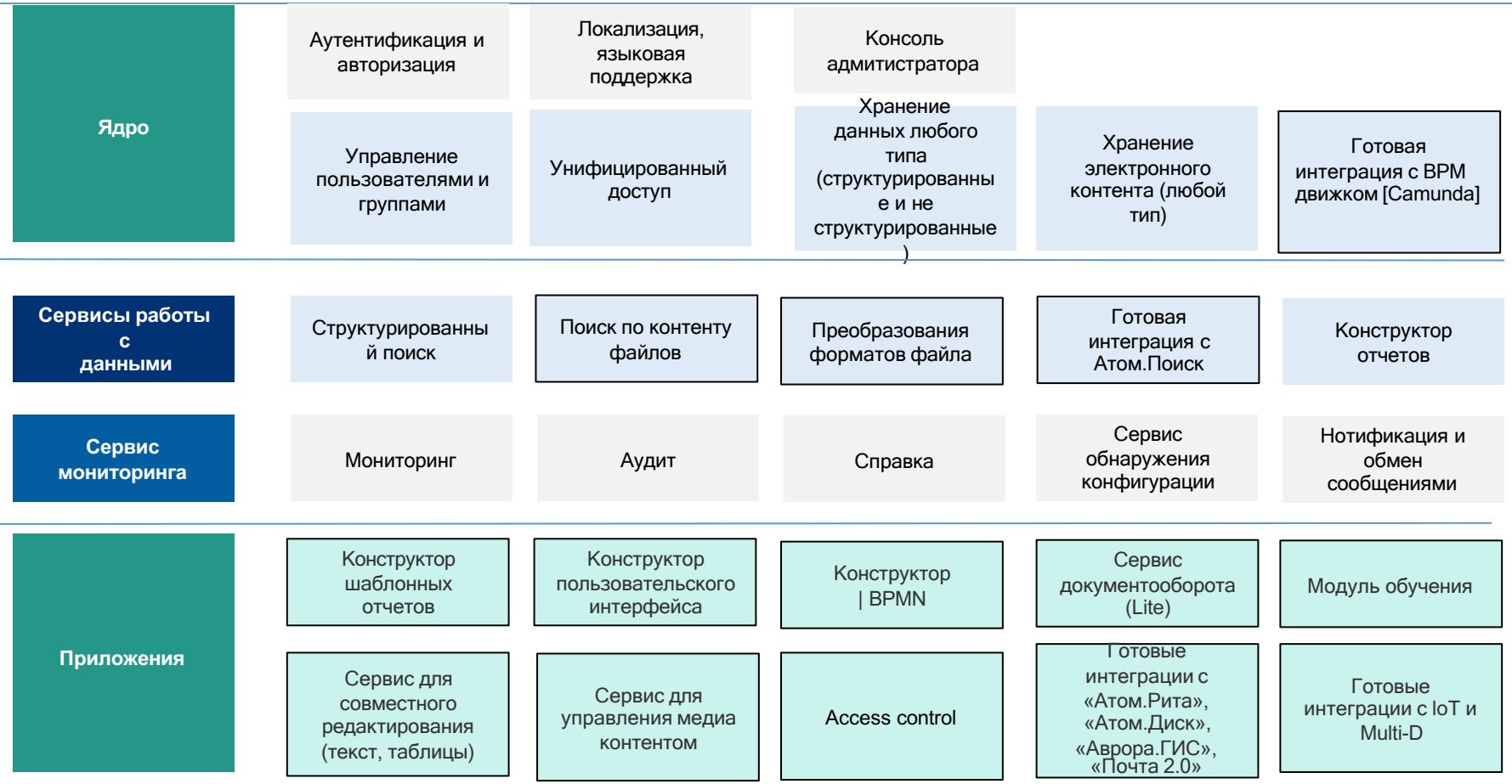


Администрирование

- В состав платформы входит подсистема журналирования и аудита, реализованная на стеке ELK.
- В состав платформы входит интерфейс администратора, реализующий основные функции управления администрирование объектов системы.



Функциональная архитектура Атом.Контент



Технологический стэк Атом.Контент

Клиентская ОС	РЕД ОС/ Астра	ОМП Аврора ОС		
Серверная ОС	РЕД ОС/ Астра			
Архитектура	Микросервисная	Клиент-серверная		
Аутентификация	LDAP	OAuth2	АтомID	RestAPI
Контейнеризация	Docker	Kubernetes (возможна)	Nomad	Consul
Разработка	Java	PostgreSQL	Springboot	React

В схеме приведеены основные технологии и отображены только самые важные.

Архитектурная реализация дистрибутива UTO

Текущий вариант Атом.Контент - набор встраиваемых библиотек Unifique Technical Objects (UTO) - предоставляющий базовые технологические сервисы по управлению данными для возможности создания на их основе конечных приложений и информационных систем

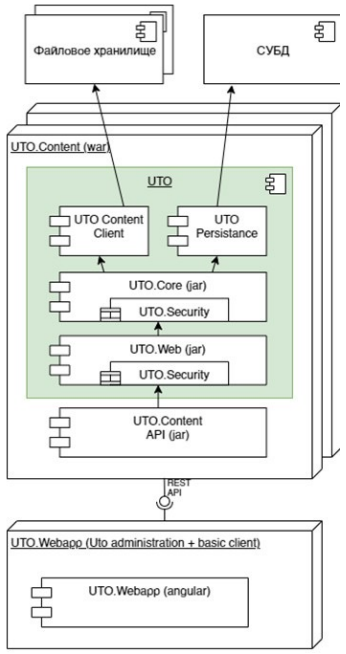
UTO.Content - это целевое приложение, основанное на библиотеках UTO, дистрибутив которого будет сертифицироваться во ФСТЭК России. Используется в качестве подсистемы Администрирования (настройка авторизации, системные настройки, работа с учетными записями и группами, работа с папками и документами, работа с аудитом)

Адаптеры - реализуют взаимодействие с файловым хранилищем и СУБД

UTO.Security - логический модуль, объединяет функции СЗИ, реализуемые в Атом.Контент

UTO.WebApp – взаимодействует с REST API и предоставляет пользовательский интерфейс к приложению UTO.Content. Используется Администраторами, может использоваться в качестве клиента для работы пользователей с базовой функциональностью для работы с документами:

- Создание;
- Размещение в папке;
- Удаление;
- Модификация;
- Назначение прав;
- Прикрепление контента.



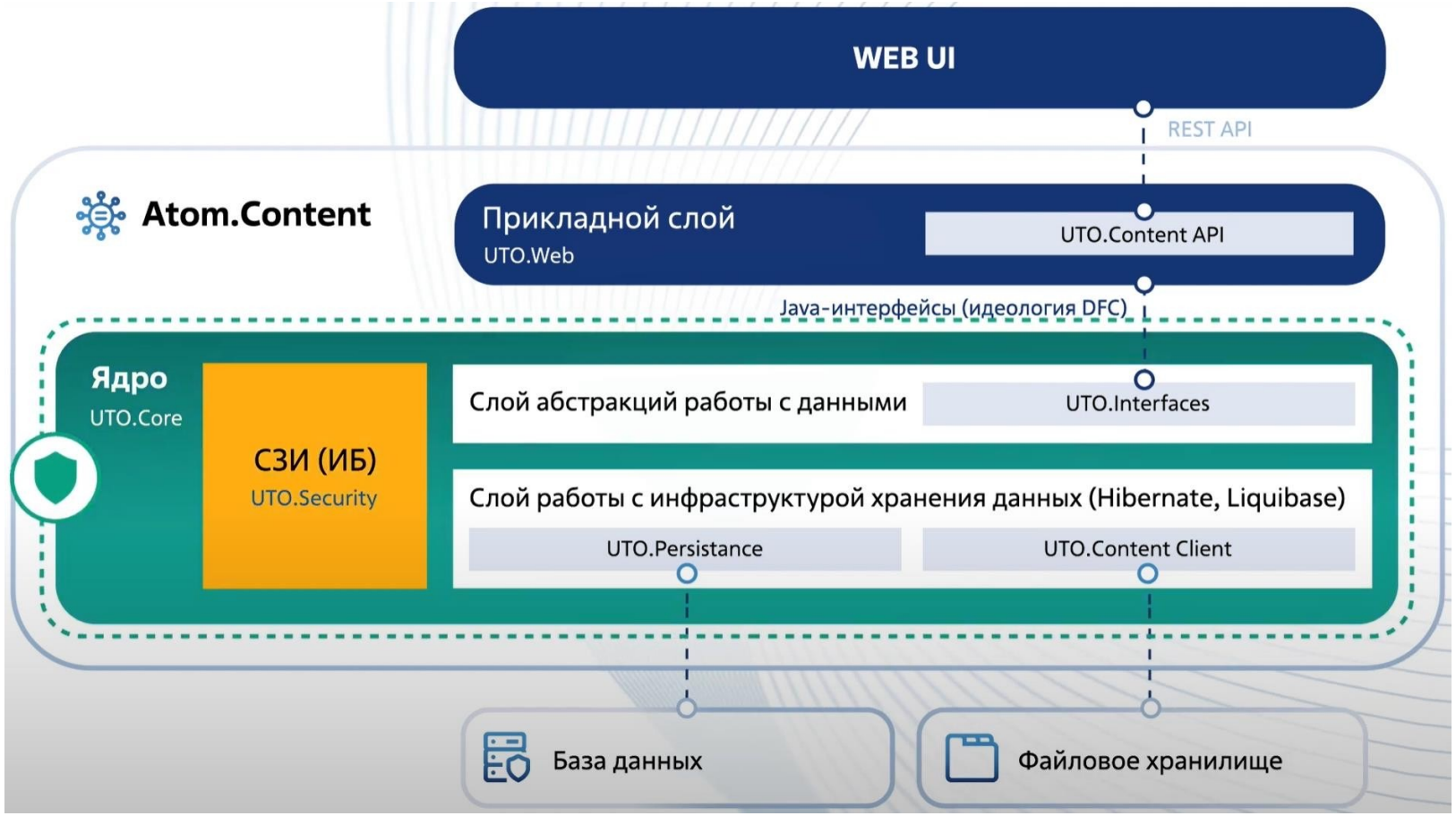
Файловое хранилище – унифицированный программный интерфейс реализующий простые CRUD-операции с файлами, вызываемый системными функциями сервиса Атом.Контент

UTO.Core – обеспечивает основные функции UTO по логике работы с мета-данными и контентом, по обеспечению безопасности, в том числе авторизации пользователя и разграничению прав доступа к данным, по взаимодействию с файловыми хранилищами и СУБД

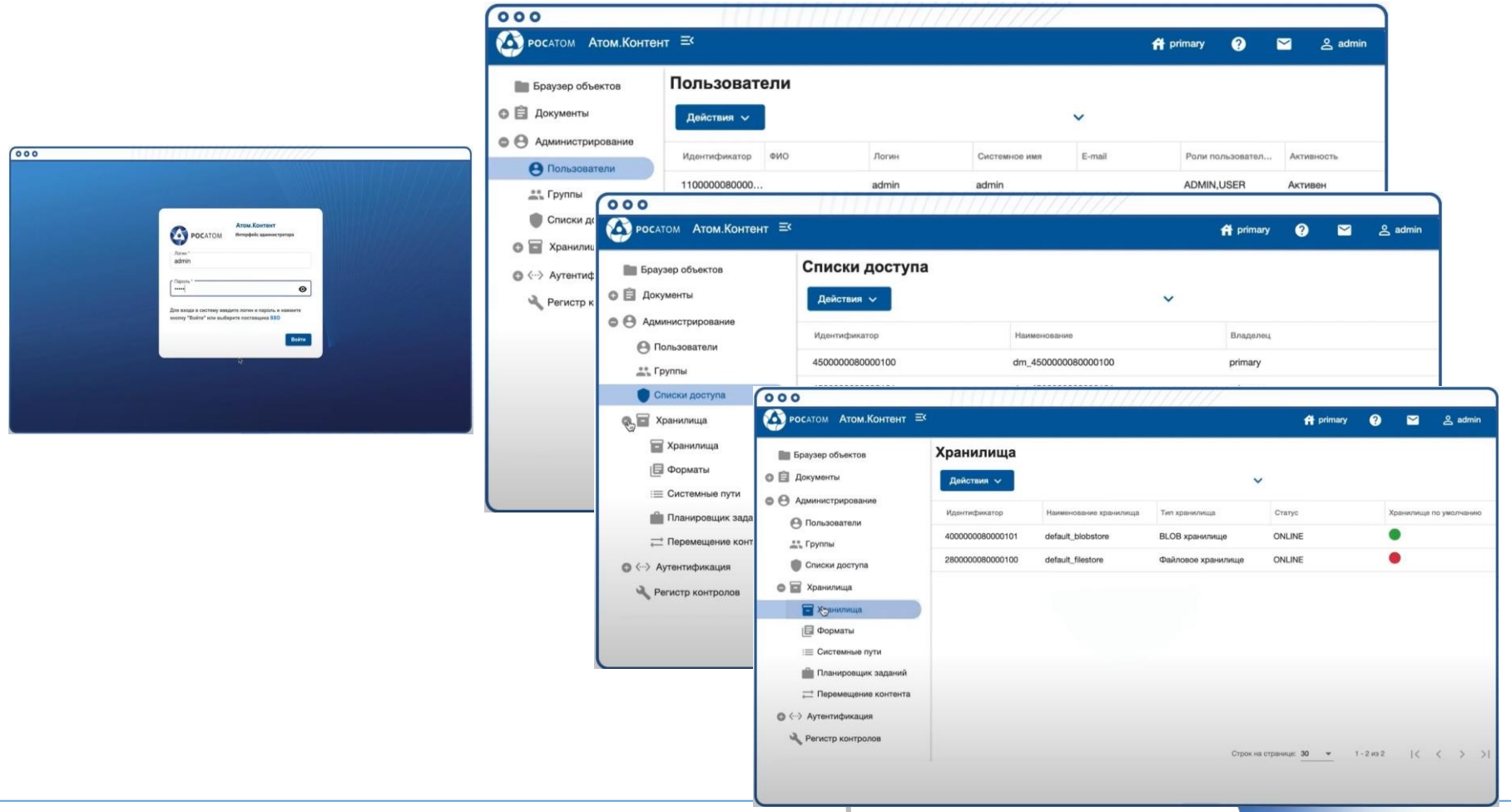
UTO.Web – обеспечивает логику взаимодействия с UTO.Core посредством web-сессий (на входе имеет http-запросы, которые библиотека оборачивает контекстом пользователя), выполняет идентификацию и аутентификацию пользователя

UTO.Content API – реализует REST API и логику работы с функциями UTO (обеспечивает работу с функциями UTO в рамках приложения UTO.Content). REST API каждого из приложений будет предоставлять только такие методы работы, которые предусмотрены в рамках данного приложения.

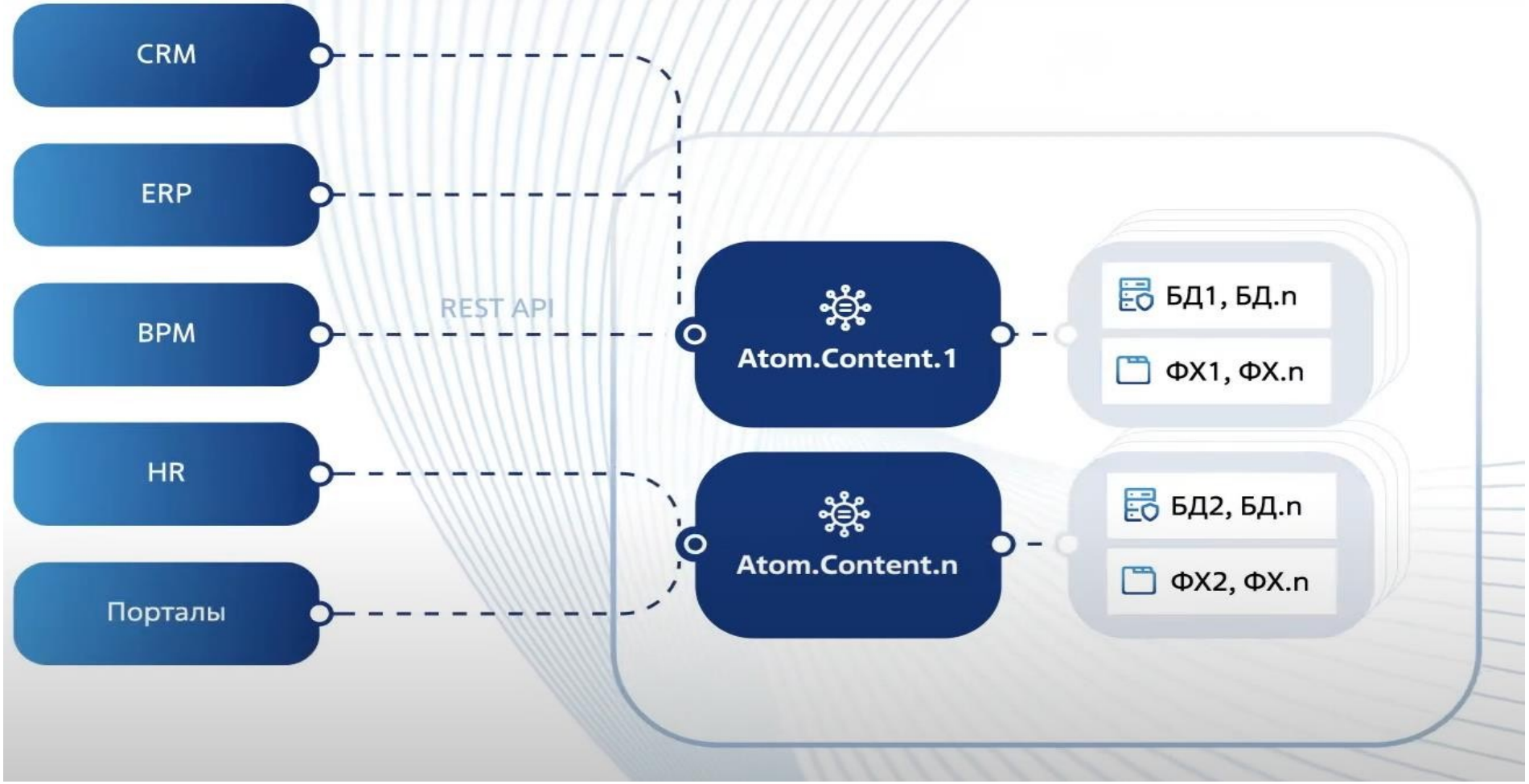
ПО Атом.Контент - собственный ИТ-продукт ГК «Росатом»



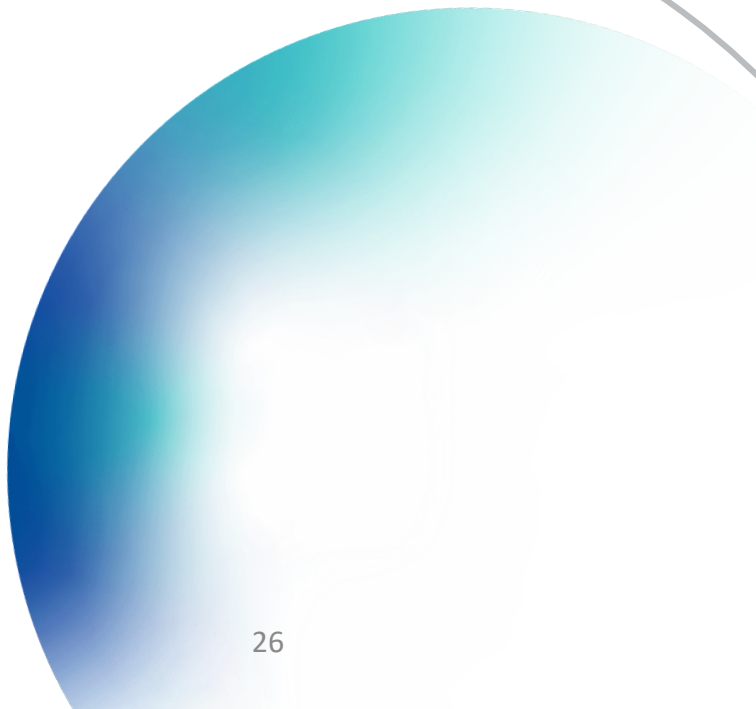
Пользовательский интерфейс и АРМ администратора



Импортозамещение ЕОСДО



Проекты на платформе Атом.Контент



Единая система электронного документооборота

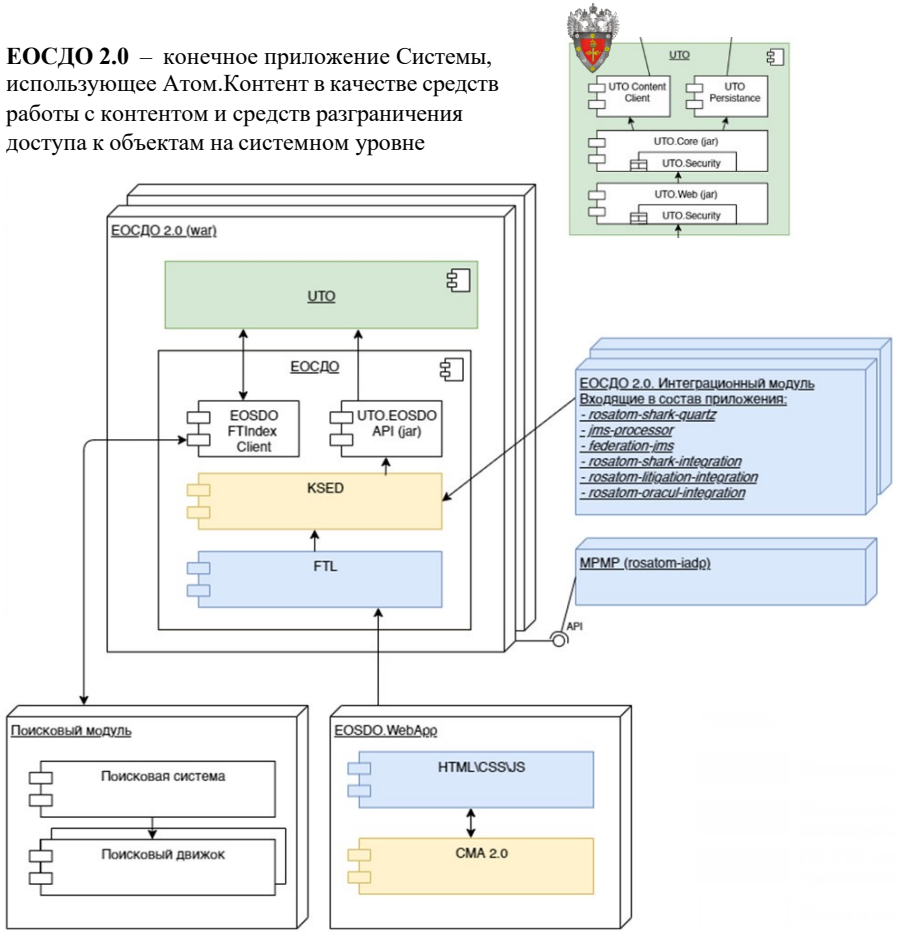


Первая в России замена западной платформы электронного документооборота российской разработкой на масштабе

110 тысяч пользователей

ЕОСДО 2.0 на основе Атом.Контент

ЕОСДО 2.0 – конечное приложение Системы, использующее Атом.Контент в качестве средств работы с контентом и средств разграничения доступа к объектам на системном уровне



UTO.EOSDO API – позволяет бизнес-слою ЕОСДО 2.0 использовать функции Атом.Контент в рамках основного приложения Системы

KSED – компонент Системы, реализующий бизнес-логику ЕОСДО

EOSDO FTIndex Client – реализует взаимодействие с поисковым модулем для преобразования и размещения поисковой информации в индексе, преобразования данных поисковых запросов субъектов к формату поисковой машины и выполнения поиска по проиндексированным объектам Системы

EOSDO.WebApp – приложение, реализующее действующий web-интерфейс к ЕОСДО 2.0 для работы бизнес-пользователей, визуально и функционально не изменяется в рамках проекта, при этом компонент WDK OpenText будет заменен

Поисковый модуль– реализует логику работы поиска, в том числе работу с индексами

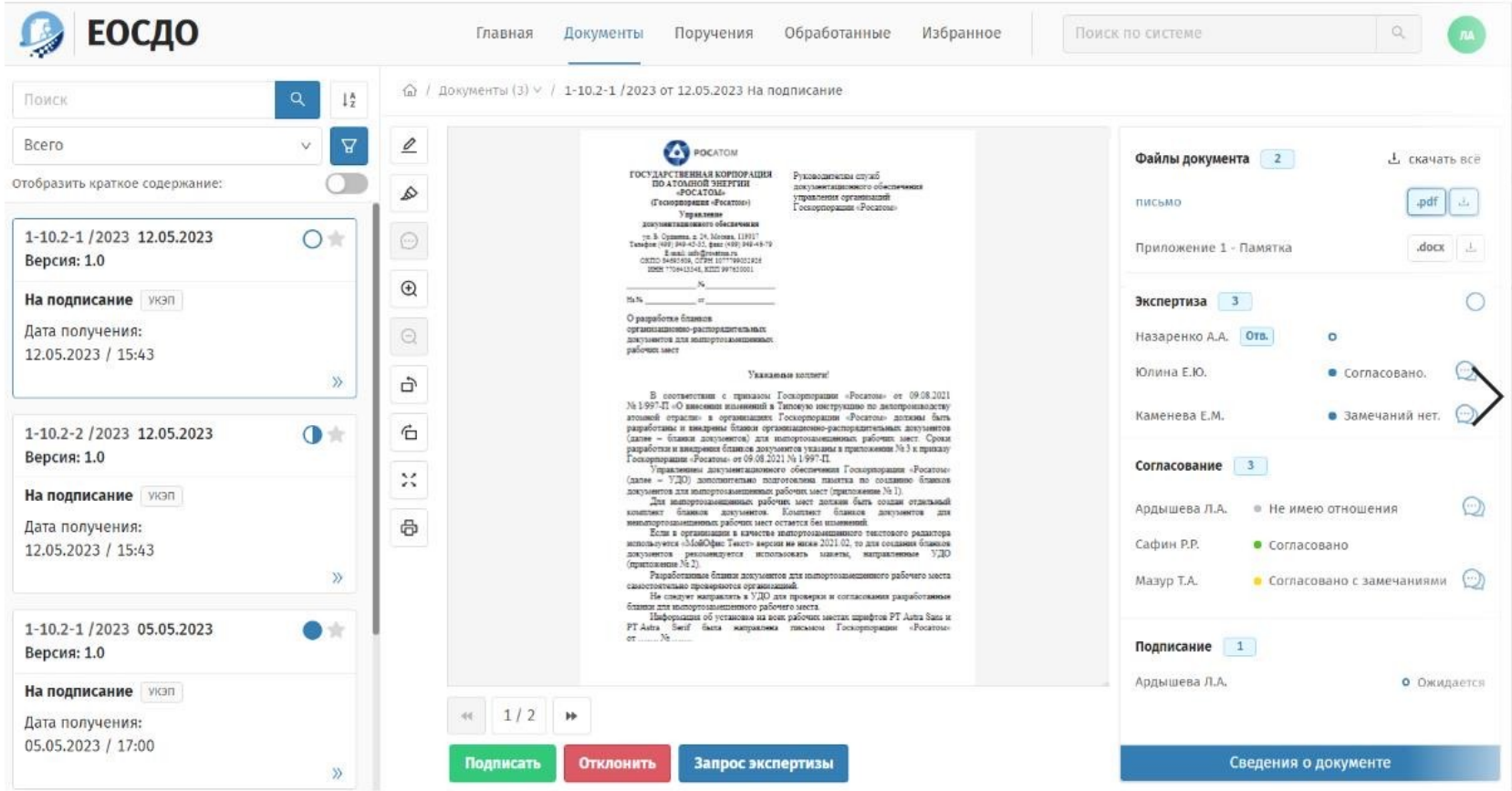
Готовые интеграции ЕОСДО

1С УРП
СУРП
ЕОС-Закупки
ЕОС НСИ
БКУ
Сириус
Монитор-Актив
РДР
Оракул
СЭП
ИСУПРИД
КХД
Litigation
ЕОС-Договор
АИС Экспертиза проектов
АИС ГСН
АИС Горячая линия
РЦК
СБиК ТСЭ
ИСУП КС
1С Цифровой Росатом
ВДНМ
СЦУДП
МЭДО
СЭД МБ

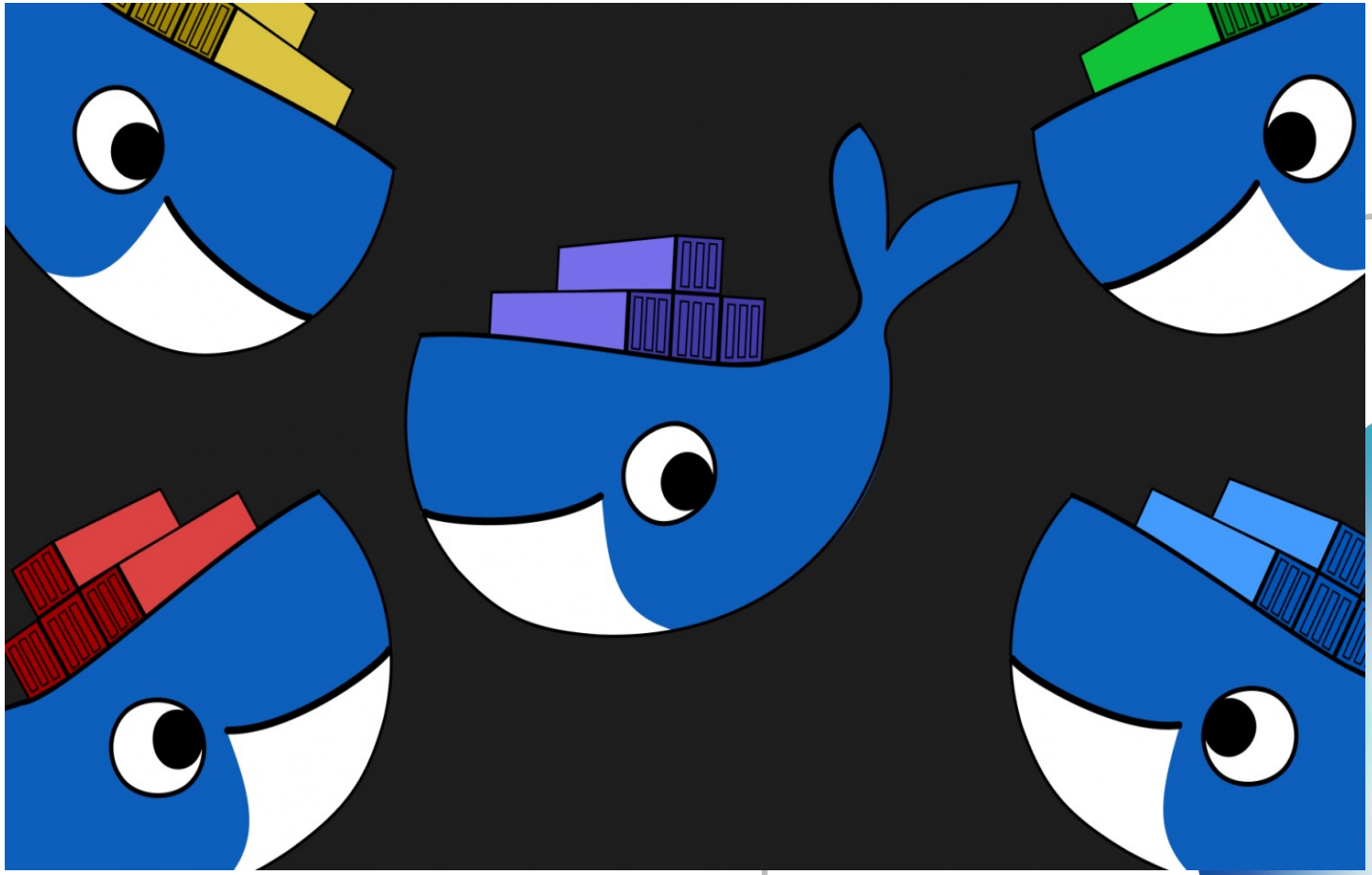
Централизованный электронный архив документов

- Три уровня хранения документов
  - до 10 лет
  - временное хранилище (для рабочих документов)
  - более 10 лет
- Версионность
- Гибкая модель управления доступами
- Оцифровка документов
- Интеграция с ЕОСДО




ЕОСДО
Главная
Документы
Поручения
Обработанные
Избранное
Поиск по системе
Поиск
Всего
Отобразить краткое содержание:
1-10.2-1 /2023 12.05.2023
Версия: 1.0
На подписание
УКЭП
Дата получения:
12.05.2023 / 15:43
1-10.2-2 /2023 12.05.2023
Версия: 1.0
На подписание
УКЭП
Дата получения:
12.05.2023 / 15:43
1-10.2-1 /2023 05.05.2023
Версия: 1.0
На подписание
УКЭП
Дата получения:
05.05.2023 / 17:00
Документы (3)
1-10.2-1 /2023 от 12.05.2023 На подписание
1 / 2
Подписать
Отклонить
Запрос экспертизы
Файлы документа 2
скачать всё
письмо
.pdf
Приложение 1 - Памятка
.docx
Экспертиза 3
Назаренко А.А.
Отв.
Юлина Е.Ю.
Согласовано.
Каменева Е.М.
Замечаний нет.
Согласование 3
Ардышева Л.А.
Не имею отношения
Сафин Р.Р.
Согласовано
Мазур Т.А.
Согласовано с замечаниями
Подписание 1
Ардышева Л.А.
Ожидается
Сведения о документе

Docker



Предыстория - все началось с виртуальных машин



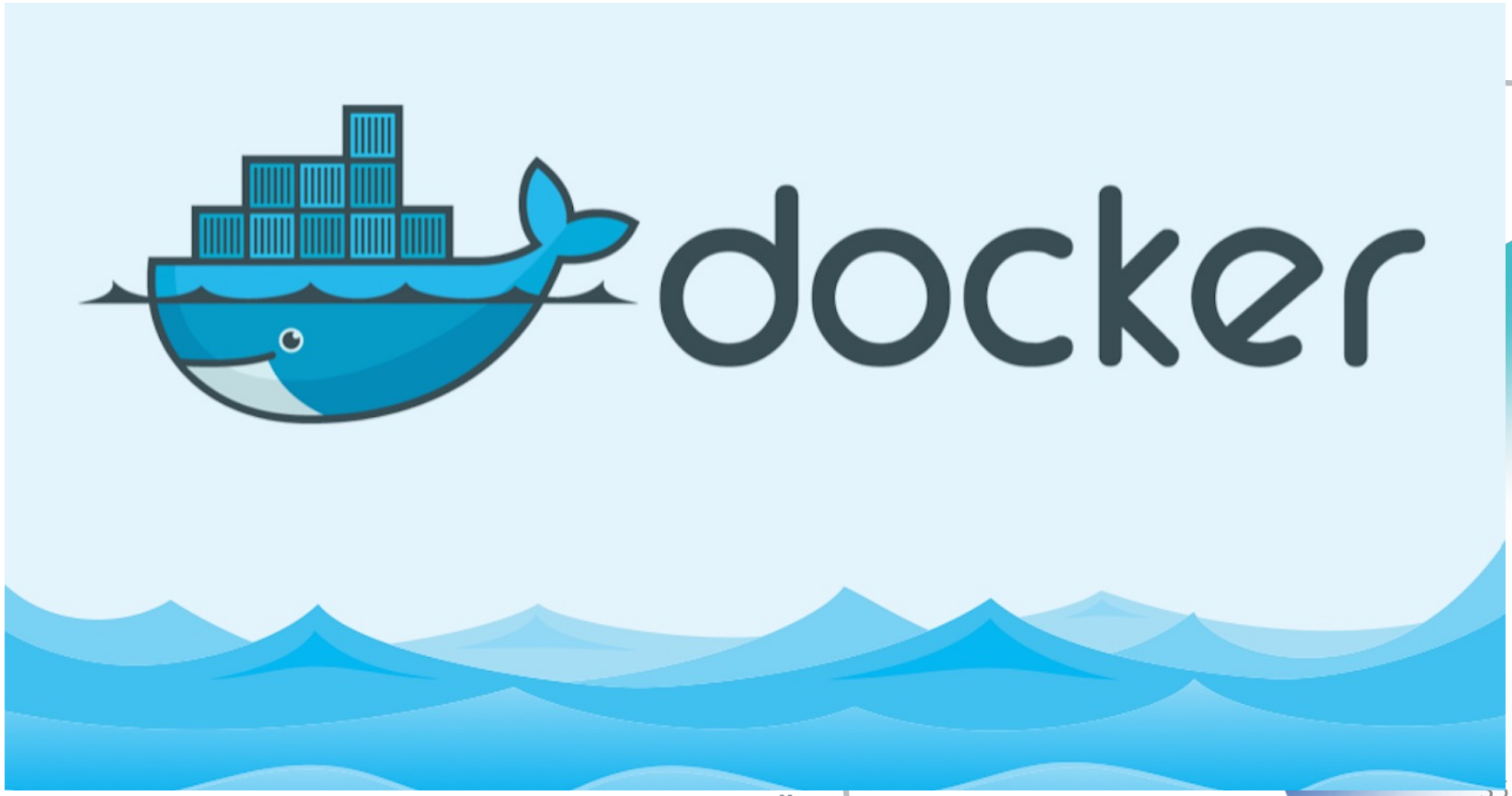
Виртуальная машина

Виртуальная машина *(часто сокращается до ВМ) мало чем отличается от физических компьютеров — ноутбука, смартфона или сервера. У нее есть ЦП, память, диски для хранения файлов и возможность подключения к Интернету. Компоненты вашего компьютера (аппаратная часть) материальны и осязаемы, тогда как виртуальные машины часто рассматриваются как виртуальные или программно-определяемые компьютеры в физических серверах, существующие только в виде кода.*

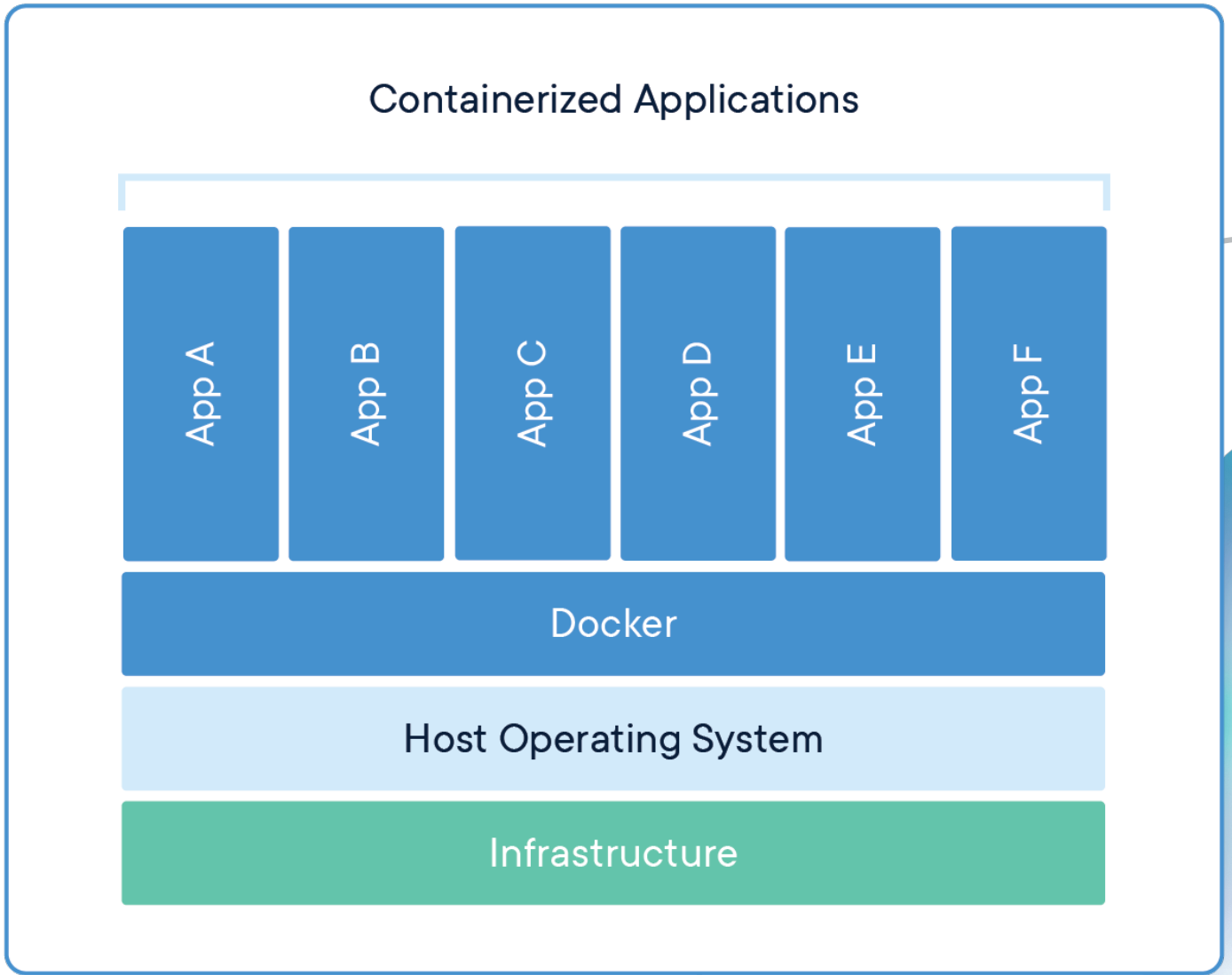
Microsoft

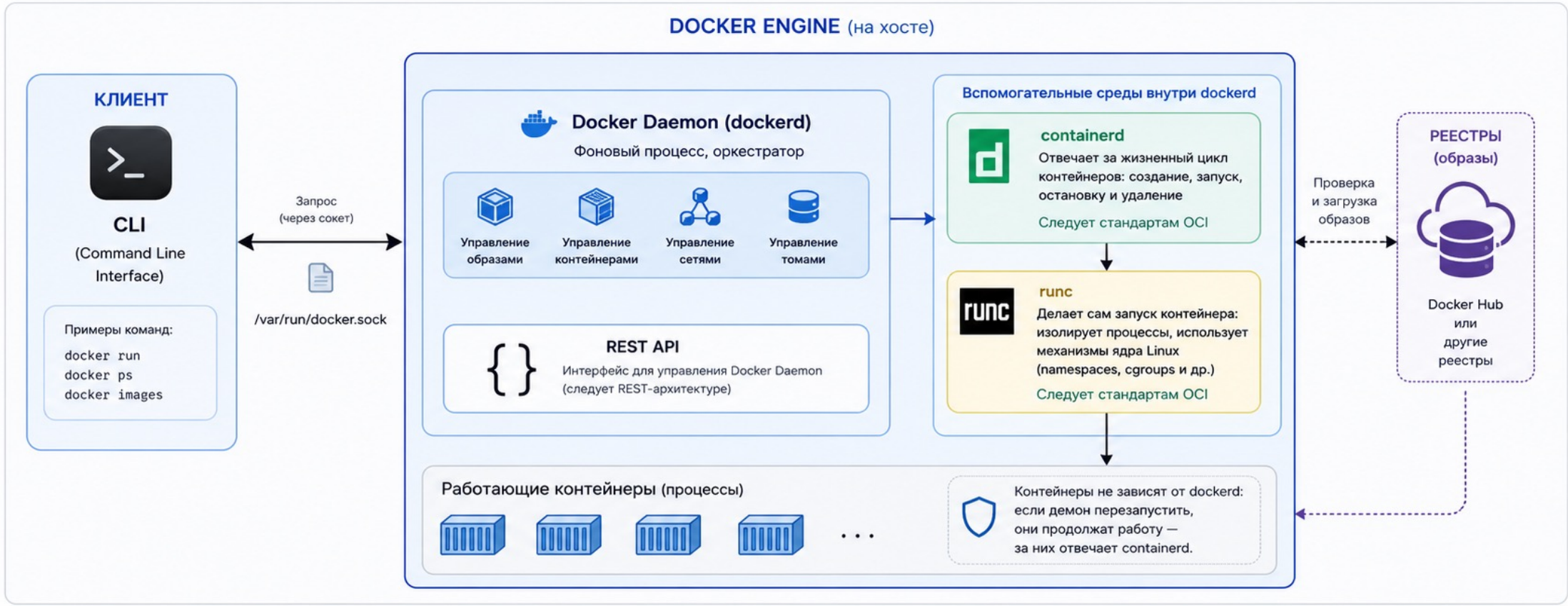
Что такое Docker?

Docker — это средство для разворачивания Docker-контейнеров, каждый из которых представляет собой отдельное приложение, запущенное в своей среде, обладающее своими сетевыми, файловыми и вычислительными ресурсами.



Как устроен Docker?




DOCKER ENGINE (на хосте)
КЛИЕНТ
CLI
(Command Line Interface)
Примеры команд:
docker run
docker ps
docker images
Запрос
(через сокет)
/var/run/docker.sock
Docker Daemon (dockerd)
Фоновый процесс, оркестратор
Управление образами
Управление контейнерами
Управление сетями
Управление томами
REST API
Интерфейс для управления Docker Daemon
(следует REST-архитектуре)
Вспомогательные среды внутри dockerd
containerd
Отвечает за жизненный цикл контейнеров: создание, запуск, остановку и удаление
Следует стандартам OCI
runc
Делает сам запуск контейнера: изолирует процессы, использует механизмы ядра Linux (namespaces, cgroups и др.)
Следует стандартам OCI
Проверка и загрузка образов
РЕЕСТРЫ (образы)
Docker Hub или другие реестры
Работающие контейнеры (процессы)
Контейнеры не зависят от dockerd: если демон перезапустить, они продолжат работу — за них отвечает containerd.


ЦЕПОЧКА ВЫПОЛНЕНИЯ КОМАНДЫ
1. CLI
Вы вводите команду, например: docker run
2. Docker Daemon (dockerd)
Получает запрос через сокет, проверяет образ, готовит окружение, сеть, тома
3. containerd
Управляет жизненным циклом контейнера (создание, запуск, остановка, удаление)
4. runc
Запускает контейнер, изолирует процессы через механизмы ядра Linux (namespaces, cgroups и др.)
5. Процессы контейнера
Ваше приложение работает внутри изолированного контейнера
Все компоненты (containerd, runc) следуют стандартам OCI — формат и запуск контейнеров совместимы между инструментами.

Контейнеры

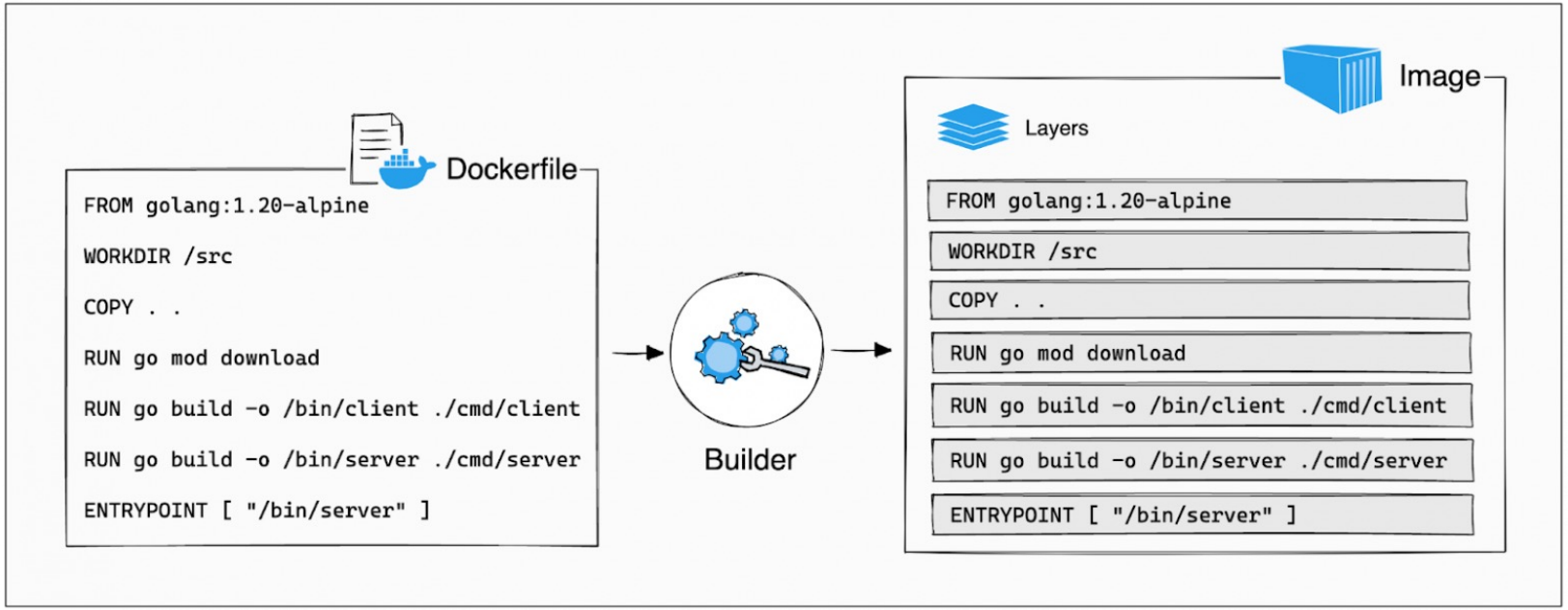
Контейнеры — это легковесные, изолированные среды выполнения, внутри которых работают приложения.

Контейнеры предоставляют множество преимуществ, включая:

- **Изоляцию.** Каждое приложение (в идеале) работает в своей собственной, изолированной среде внутри контейнера;
- **Повторяемость.** Docker гарантирует, что контейнер, который работает на машине разработчика, будет также работать и на сервере, без каких-либо неожиданностей;
- **Простоту доставки.** Образы могут быть легко перенесены между различными окружениями, будь то локальное окружение, тестовые сервера или облачная инфраструктура.

Образы

Образ Docker — это статичное описание содержимого контейнера, включающее в себя все зависимости, настройки окружения, библиотеки и бинарные файлы, необходимые для выполнения приложения.



Запуск первого контейнера с Docker

Предположим, что вы уже установили Docker CLI или Docker Desktop для своей системы и, возможно, попытались запустить свой первый hello world контейнер командой docker run hello-world.

Разберем подробнее, какие действия выполняет эта команда:

- Docker ищет образ hello-world в локальном хранилище. Если образ не найдется, то Docker скачает его с Docker Hub;
- Далее Docker создает контейнер на основе этого образа и запускает его;
- Контейнер выполняет скрипт, который выводит на экран приветственное сообщение и завершает работу.

Основные команды

Запуск контейнера — это основное действие, которое вы будете выполнять в Docker.

Теперь попробуем запустить более сложное приложение. Например, официальный образ операционной системы Ubuntu: docker run -it ubuntu bash

Эта команда делает следующее:

- флаг --it объединяет две опции: -i (interactive) и -t (tty)
- -i (--interactive) означает, что запускаемый контейнер будет получать стандартный поток ввода с хоста и направлять его в приложение, работающее в контейнере. По-умолчанию контейнеры стартуют изолированно, и stdin запущенного приложения не имеет связи с внешним миром.
- -t (--tty) указывает докеру создать для запущенного приложения псевдотерминал, что позволит удобно работать с ним из вашего терминала.
- ubuntu — название образа, который мы запускаем Ubuntu;
- bash — команда внутри контейнера ubuntu.

Основные команды

Чтобы остановить контейнер, используется команда: docker stop <container_id>

- Где <container_id> — это идентификатор контейнера, который вы хотите остановить. Вы можете определить идентификатор контейнера с помощью следующей команды: docker ps. Эта команда выводит список запущенных контейнеров вместе с их идентификаторами.
- Если вам нужно перезапустить контейнер, используйте команду docker restart: docker restart <container_id>. ID контейнера можно получить из вывода команды ps, однако большинство команд, работающих с ID контейнеров, могут работать и с названиями.
- Чтобы удалить контейнер, необходимо сначала остановить его, а затем использовать команду rm: docker rm <container_id>
- Для одновременной остановки и удаления контейнера можно использовать флаг -f (force): docker rm -f <container_id>

Управление образами Docker

Для загрузки образа без его запуска можно использовать команду pull, например, docker pull ubuntu. Эта команда загрузит в локальное хранилище последний (latest) образ Ubuntu, однако при необходимости можно указать конкретную версию образа: docker pull ubuntu:20.04

Чтобы увидеть все доступные на вашем компьютере образы, используйте команду: docker images

Для удаления образа используйте команду docker rmi (remove image): docker rmi <image_id>

Получить идентификатор образа можно с помощью команды docker images.

Dockerfile – инструкции для создания образа

Dockerfile — это файл с инструкциями для сборки Docker-образа. В нём указывают базовый образ, рабочую папку, команды запуска, переменные окружения, копируемые файлы и используемые порты.

Основные команды:

- FROM — выбирает базовый образ.
- WORKDIR — задаёт рабочую директорию.
- RUN — выполняет команды во время сборки.
- CMD и ENTRYPOINT — определяют запуск контейнера. ENTRYPOINT обычно сложнее переопределить.
- COPY и ADD — добавляют файлы в образ, при этом ADD также умеет загружать их из сети.
- ARG и ENV — задают переменные для сборки и работы контейнера.
- LABEL, MAINTAINER, EXPOSE — добавляют информацию об образе и портах.
- EXPOSE только указывает используемый порт, но не открывает его автоматически.

Простой пример Dockerfile

```
# Указываем базовый образ
FROM node:14
# Устанавливаем рабочую директорию внутри будущего контейнера
WORKDIR /app
# Копируем package.json и package-lock.json в /app (./ из-за WORKDIR)
COPY package*.json ./
# Устанавливаем зависимости
RUN npm install
# Копируем файлы приложения (с хоста (контекст) в образ (/app))
COPY . .
# Открываем порт
EXPOSE 3000
# Запускаем приложение
CMD ["node", "server.js"]
```

Поэтапная (multistage) сборка

multistage -сборка позволяет уменьшить размер итоговых образов, используя несколько команд FROM.

В качестве примера рассмотрим сборку простого Go-приложения:

```
# BUILD STAGE
FROM golang:1.16 AS build
WORKDIR /go/src/app
COPY . .
RUN go build -o myapp

# RUN STAGE
FROM alpine:latest
WORKDIR /root/
COPY --from=build /go/src/app/myapp .
CMD ["./myapp"]
```

Docker Hub, репозитории образов

Docker Hub — это репозиторий, который предоставляет разработчикам возможность легко обмениваться и управлять контейнерными образами.

С помощью Docker Hub вы можете:

- Искать и загружать публичные образы, предоставляемые сообществом;
- Создавать и делиться собственными изображениями;
- Управлять автоматическими сборками и интеграциями с системой контроля версий.

Сети

- Работа с сетями — это одна из ключевых составляющих контейнеризации в Docker. Отсутствие настройки сетевого взаимодействия контейнеров может привести к проблемам с доступом к вашим сервисам.
- Docker предоставляет несколько драйверов сетевого взаимодействия, из которых наиболее распространённые — bridge, host и overlay.

Сети. Bridge

- Этот сетевой режим используется по умолчанию. В нем создается виртуальный мост (bridge), который позволяет контейнерам общаться друг с другом и с хост-машиной.

- Для подключения контейнера к сети, укажите имя сети при запуске контейнера с использованием флага --network

```
docker network create --driver bridge app_network
```

```
docker run -d --network app_network --name app nginx
```

Сети. Host

В этом режиме контейнер использует сетевой стек хост-машины. Это означает, что контейнер и хост имеют общий IP-адрес и порты. Host-сеть полезна для уменьшения сетевой задержки, однако она уменьшает изоляцию между контейнером и хостом.

docker run -d --network host nginx

Сети. Overlay

- Этот режим в основном используется в кластерных средах и Docker Swarm.
- Overlay-сети позволяют контейнерам, работающим на разных физических или виртуальных машинах, общаться друг с другом так, будто они находятся на одной сети. Это достигается путем создания распределенной сети поверх существующей физической инфраструктуры.
- docker network create --driver overlay --subnet 10.0.9.0/24 my_overlay_network
- Для списка доступных сетей используйте команду: docker network ls
- Для отключения контейнера от сети используйте команду: docker network disconnect <network_name> <container_id>
- Чтобы удалить сеть, используйте команду: docker network rm <network_name>

Docker Volumes и связывание контейнера с файловой системой хоста (bind mounts)

- Volumes и bind mounts — два ключевых механизма для работы с данными в контейнерах. Они необходимы, чтобы эффективно управлять данными, обеспечивать их сохранность и доступность.
- Docker volumes существуют, чтобы хранить данные отдельно от контейнера. Даже в случае, если контейнер удалится, данные, хранящиеся в volume, останутся нетронутыми, что важно, когда проект уже развернут на площадке.
- Bind Mounts немного отличаются от volumes. Этот подход представляет собой простое монтирование директорий с хоста в директории внутри контейнера. Это позволяет контейнерам иметь прямой доступ к данным на хосте, что удобно для среды разработки и тестирования.

Docker Volumes и связывание контейнера с файловой системой хоста (bind mounts)

- Можно заметить, что volume и bind mounts отличаются только типом и значением src. В случае с volumes вы указываете название тома, а в случае с bind mounts указывается путь на хосте, который нужно опрокинуть в контейнер.

	Volumes	Bind mounts
Путь на хосте	Выбирает Docker	Указывается разработчиком
Создает новый volume	Да	Нет
Поддерживает драйверы volumes	Да	Нет

Docker Volumes и связывание контейнера с файловой системой хоста (bind mounts)

- Для создания volumes и bind mount также может использоваться следующий синтаксис команды docker run: docker run -d -v /path/on/host:/path/in/container my_image в случае использования bind mounts.
- Или docker volume create my_volume && docker run -d -v my_volume:/data my_image в случае использования томов.
- Теперь данные по пути /data внутри контейнера будут храниться в my_volume. Volume можно отключать, заменять и делать еще много всего. Разово создав volume, пересоздавать его не нужно.

Docker Compose

Docker Compose — это мощный инструмент, разработанный для упрощения работы с многоконтейнерными приложениями. Docker Compose позволяет вам описать и запустить сложные приложения, состоящие из нескольких контейнеров, с минимальными усилиями. В этом разделе мы погрузимся в основы Docker Compose и его применение.

Основные возможности Docker Compose включают:

- Декларативное описание сервисов, volumes и networks в формате yaml;
- Управление всеми службами, указанными в конфигурационном файле, при помощи единой утилиты docker compose;
- Управление жизненным циклом контейнеров.

Docker Compose

```yaml
services:
    web:
        image: nginx:latest
        ports:
            - "8000:80"
        networks:
            - app-network
    app:
        build:
            args:
                user: www-data
                uid: 33
                app_mode: development
            context: .
            dockerfile: Dockerfile
        restart: always
        image: app
        container_name: app
        working_dir: /var/www/
        volumes:
            -'./:/var/www'
        networks:
            - app-network
    db:
        image: mysql:latest
        volumes:
            -'app-db:/var/lib/mysql'
        environment:
            DB_PASSWORD: password
        networks:
            - app-network
networks:
    app-network:
        name: app-network
        driver: bridge
volumes:
    app-db:
        driver: local
```

Управление контейнерами

Для управления отдельными службами Docker Compose предоставляет удобные команды.

Просмотр списка запущенных контейнеров: docker compose ps

Просмотр всех контейнеров: docker compose ps -a

Просмотр логов приложения: docker compose logs <container_name>

Перезапуск контейнера: docker compose restart <container_name>

Docker-команды для работы с образами

- docker images — просмотр локально сохранённых образов;
- docker rmi — удаление образов;
- docker inspect — просмотр манифеста образа;
- docker save / docker load — экспорт и импорт образов в виде tar-архивов;
- docker build — сборка образа по Dockerfile;
- docker tag — присвоение образу символического имени;
- docker pull — загрузка образа из удалённого репозитория;
- docker push — загрузка локального образа в удалённый репозиторий.

Команды для работы с контейнерами

- docker run — создать и запустить контейнер;
- docker start / docker stop — запустить или остановить контейнер;
- docker ps — показать список контейнеров;
- docker exec — выполнить команду внутри контейнера;
- docker rm — удалить контейнер;
- docker logs — просмотреть логи контейнера;
- docker stats — аналог htop для контейнеров;
- docker cp — скопировать файлы внутрь или из контейнера.

Docker Compose предоставляет следующие команды

- docker compose up / docker compose up -d — запустить или обновить все сервисы, флаг -d указывает, что запуск надо выполнять в фоне;
- docker compose ps — показать список запущенных контейнеров;
- docker compose logs — посмотреть логи сервисов;
- docker compose down — остановить и удалить контейнеры;
- docker compose rm — удалить контейнер.

Ограничения Docker Compose

Несмотря на удобство Docker Compose, у него есть ограничения.

- Он не поддерживает балансировку нагрузки и не умеет запускать контейнеры на разных серверах.
- Проверка состояния (health checks) ограничена, и для автоматического восстановления требуется использовать сторонние инструменты
- Конструкция depends_on не учитывает задержку запуска сервисов внутри контейнеров, она ждёт, когда запустится контейнер, а не сервис в нём.

Спасибо за внимание!